



SZTUCZNA INTELIGENCJA I INŻYNIERIA WIEDZY

•

LISTA 3

Zofia Różańska, 280526



13 MAJA 2026

Spis treści

1	Wstęp	2
2	Środowisko	2
3	Zadanie 1	2
3.1	Opis problemu	2
3.2	Wykorzystane rozszerzenia PDDL	2
3.3	Hierarchia typów	3
3.4	Graf sieci transportowej	4
3.5	Opis domeny	4
3.6	Kod	5
3.7	Wygenerowany plan	5
3.8	Analiza planu	6
3.9	Problemy implementacyjne	6
4	Zadanie 2	7
4.1	Opis problemu	7
4.2	Hierarchia obiektów	7
4.3	Kod	8
4.4	Uruchomione planery	8
4.5	Wyniki eksperymentów	8
5	Zadanie 3	9
5.1	Opis problemu	9
5.2	Hierarchia obiektów	9
5.3	Uruchomione planery	9
5.4	Wyniki eksperymentów	10
6	Podsumowanie	11
7	Materiały źródłowe	11

1. Wstęp

Niniejsze sprawozdanie zawiera rozwiązania trzech problemów planowania z wykorzystaniem języka **PDDL** (Planning Domain Definition Language). Celem ćwiczenia było zapoznanie się z podstawami modelowania problemów planowania oraz działaniem automatycznych planerów.

Zadania dotyczą problemów: transportu paczek, sprzątanía pokoi przez robota oraz przenoszenia piłek między pokojami przez robota z dwoma ramionami. Dla każdego problemu przygotowałam pliki `domain.pddl` oraz `problem.pddl`, a następnie uruchomiłam planery w celu wygenerowania planu działania.

2. Środowisko

Do realizacji zadań wykorzystałam środowisko Visual Studio Code wraz z rozszerzeniem *PDDL*¹ wspierającym język PDDL. Rozszerzenie ułatwia edycję plików PDDL, a także pozwala na uruchamianie planerów dostępnych w usłudze *Planner as a Service*. Podczas wykonywania zadań korzystałam z kilku różnych planerów.

3. Zadanie 1

3.1. Opis problemu

Celem zadania było zamodelowanie problemu transportu paczek z wykorzystaniem języka PDDL z rozszerzeniami. W mojej wersji problem obejmuje transport pięciu paczek pomiędzy sześcioma różnymi miastami z użyciem trzech różnych środków transportu: ciężarówek, samolotów oraz statku.

Domena uwzględnia trzy różne typy lokalizacji: miasta, lotniska oraz porty. Rozróżnialne są także trzy typy połączeń: drogowe, lotnicze i wodne. Każdy środek transportu może poruszać się wyłącznie w ramach odpowiadającemu mu rodzajowi połączeń.

Moja implementacja stosuje mechanizm kosztów działań, co pozwala planerowi na minimalizację całkowitego kosztu dostarczenia wszystkich paczek.

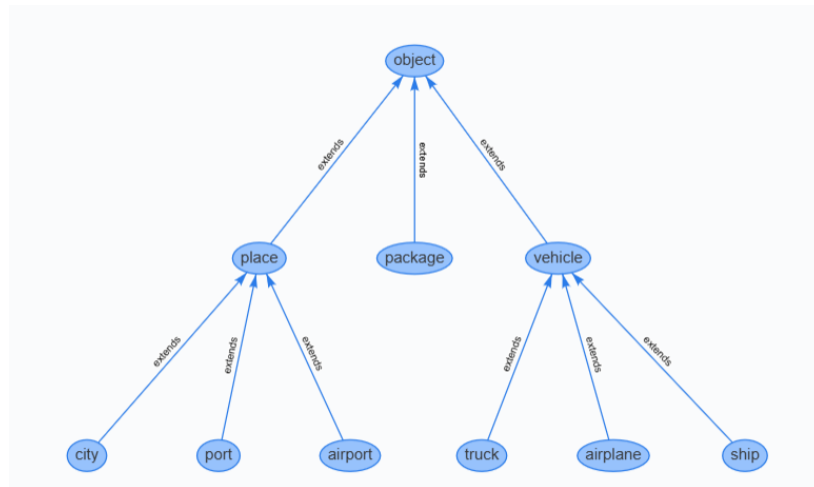
3.2. Wykorzystane rozszerzenia PDDL

W modelu wykorzystałam następujące rozszerzenia języka PDDL:

- `:typing` – typowanie obiektów,
- `:strips` – podstawowy model STRIPS,
- `:numeric-fluents` – funkcje numeryczne,
- `:action-costs` – koszty działań.

¹<https://marketplace.visualstudio.com/items?itemName=jan-dolejsi.pddl>

3.3. Hierarchia typów

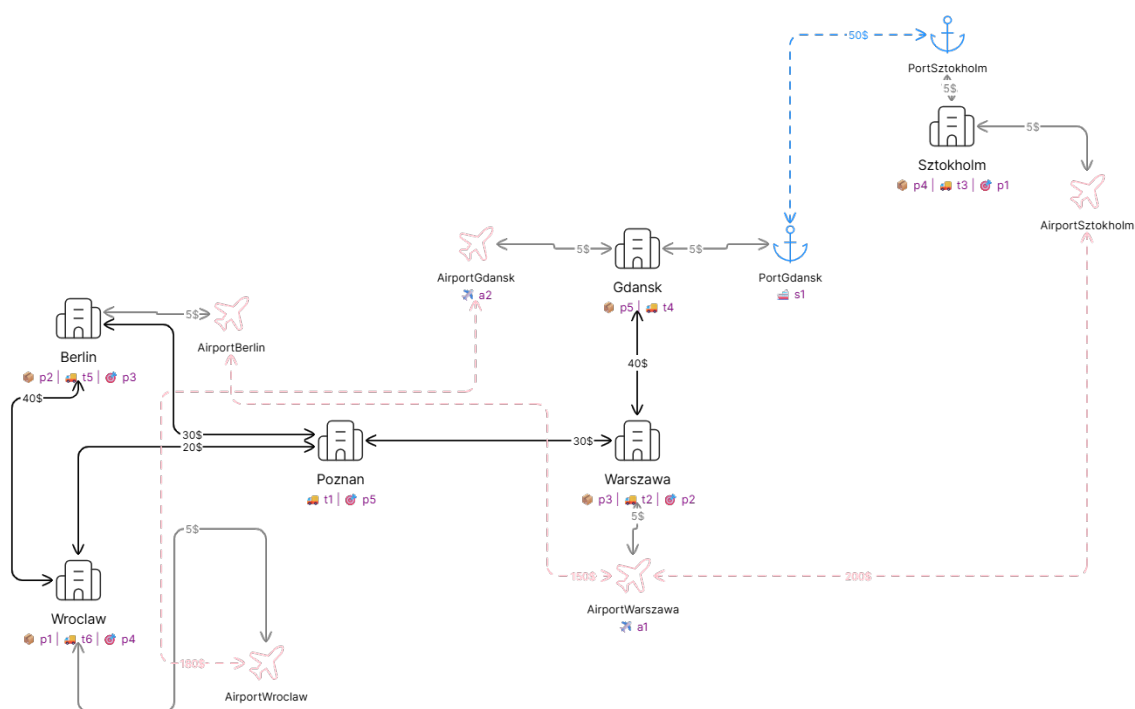


Rysunek 1: Hierarchia typów w domenie transportowej

3.4. Graf sieci transportowej

Poniższy schemat odzwierciedla:

- rozważane lokalizacje,
- połączenia transportowe,
- koszt pokonania tras,
- lokalizacje nadania paczek,
- lokalizacje docelowe paczek



eraser

Rysunek 2: Graf połączeń transportowych

3.5. Opis domeny

Plik domenowy definiuje trzy typy pojazdów:

- **truck** – transport drogowy,
- **airplane** – transport lotniczy,
- **ship** – transport wodny.

Każdy pojazd posiada własne akcje przemieszczania:

- **drive-truck**,

- fly-airplane,
- sail-ship.

Połączenia pomiędzy lokalizacjami zostały podzielone na:

- połączenia drogowe – link-road,
- połączenia lotnicze – link-air,
- połączenia wodne – link-water.

Transport paczek realizowany jest poprzez akcje:

- load – załadunek paczki z lokalizacji do pojazdu,
- unload – rozładunek paczki z pojazdu do lokalizacji.

Funkcja `total-cost` przechowuje całkowity koszt wykonanych działań. Koszty zależą od rodzaju transportu oraz odległości pomiędzy lokalizacjami.

3.6. Kod

Pliki `domain.pddl` i `problem.pddl` znajdują się w folderze `transport`.

3.7. Wygenerowany plan

Do rozwiązywania problemu używałam planerów OPTIC oraz ENHSP, jednak ENHSP zwrócił rozwiązanie lepszej jakości (35 vs 55 kroków). Poniżej znajduje się plan wygenerowany przez ENHSP.

```
;;!domain: logistics
;;!problem: transport-5-packages-advanced

0.00000: (load t3 p4 Sztokholm)
1.00000: (load t5 p2 Berlin)
2.00000: (drive-truck t5 Berlin Poznan)
3.00000: (drive-truck t5 Poznan Warszawa)
4.00000: (load t2 p3 Warszawa)
5.00000: (drive-truck t2 Warszawa Poznan)
6.00000: (drive-truck t2 Poznan Berlin)
7.00000: (load t4 p5 Gdansk)
8.00000: (drive-truck t4 Gdansk Warszawa)
9.00000: (drive-truck t4 Warszawa Poznan)
10.00000: (load t6 p1 Wroclaw)
11.00000: (drive-truck t6 Wroclaw Poznan)
12.00000: (drive-truck t6 Poznan Warszawa)
13.00000: (drive-truck t6 Warszawa Gdansk)
14.00000: (drive-truck t3 Sztokholm PortSztokholm)
15.00000: (unload t3 p4 PortSztokholm)
```

```
16.00000: (unload t5 p2 Warszawa)
17.00000: (unload t4 p5 Poznan)
18.00000: (unload t2 p3 Berlin)
19.00000: (drive-truck t6 Gdansk PortGdansk)
20.00000: (unload t6 p1 PortGdansk)
21.00000: (load s1 p1 PortGdansk)
22.00000: (sail-ship s1 PortGdansk PortSztokholm)
23.00000: (load s1 p4 PortSztokholm)
24.00000: (unload s1 p1 PortSztokholm)
25.00000: (sail-ship s1 PortSztokholm PortGdansk)
26.00000: (unload s1 p4 PortGdansk)
27.00000: (load t6 p4 PortGdansk)
28.00000: (drive-truck t6 PortGdansk Gdansk)
29.00000: (drive-truck t6 Gdansk Warszawa)
30.00000: (drive-truck t6 Warszawa Poznan)
31.00000: (drive-truck t6 Poznan Wroclaw)
32.00000: (unload t6 p4 Wroclaw)
33.00000: (load t3 p1 PortSztokholm)
34.00000: (drive-truck t3 PortSztokholm Sztokholm)
35.00000: (unload t3 p1 Sztokholm)

; Makespan: 35
; Metric: 35
```

3.8. Analiza planu

Planer wykorzystuje równoległość działań, realizując jednocześnie transport różnych paczek przy użyciu wielu pojazdów.

W rozwiązaniu ENHSP wykorzystane zostały tylko dwa dostępne środki transportu – ciężarówki i statek. Co ciekawe, w planie OPTIC również zostały wykorzystane tylko dwa środki transportu, lecz tutaj planer wykorzystał ciężarówki oraz samoloty. Pokazuje to, że mechanizmy wewnątrz różnych planerów znacząco się od siebie różnią.

Dzięki zastosowaniu funkcji kosztów planer stara się minimalizować całkowity koszt wykonania planu. W rezultacie wybierane są trasy oraz środki transportu pozwalające osiągnąć kompromis pomiędzy długością planu a kosztem transportu.

3.9. Problemy implementacyjne

Na początku planowałam zaimplementować model uwzględniający zarówno funkcje kosztów, jak i zależności czasowe przy użyciu `:durative-actions`. Napotkałam jednak na pewne ograniczenia planera OPTIC. Próbowałam obejść te ograniczenia, lecz niestety nie udało mi się.

Ostatecznie zdecydowałam się na uproszczenie modelu. Zrezygnowałam z akcji czasowych na rzecz klasycznych `:action`, zachowując optymalizację kosztu jako jedyne kryterium. Takie rozwiązanie było łatwiejsze w implementacji oraz pozwalało testować różne planery.

4. Zadanie 2

4.1. Opis problemu

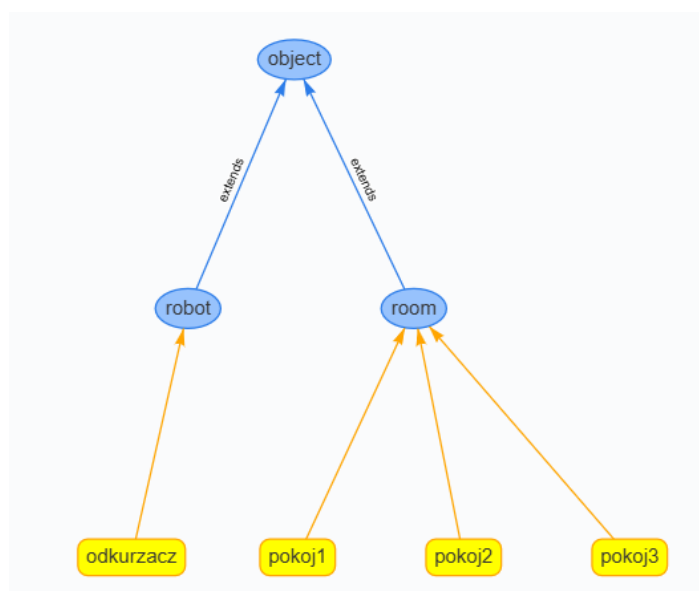
Celem zadania było zamodelowanie problemu planowania, w którym robot ma odwiedzić wszystkie pokoje i je odkurzyć. Robot może przemieszczać się pomiędzy pokojami oraz wykonywać akcję sprzątania w pokoju, w którym aktualnie się znajduje.

W domenie, zgodnie z poleceniem, wykorzystałam następujące elementy:

- obiekty: *robot*, *pokoj1*, *pokoj2*, *pokoj3*
- predykaty:
 - (at ?r - robot ?p - room)
 - (dirty ?p - room)
 - (clean ?p - room)
- akcje:
 - move
 - clean

Zadaniem planera było wygenerowanie sekwencji działań prowadzącej do osiągnięcia stanu, w którym wszystkie pokoje są czyste.

4.2. Hierarchia obiektów



Rysunek 3: Hierarchia obiektów w zadaniu 2.

4.3. Kod

Pliki `domain.pddl` i `problem.pddl` znajdują się w folderze `room-duster`.

4.4. Uruchomione planery

Uruchomiłam model z wykorzystaniem trzech różnych planerów:

- BFWS – FF-parser version,
- LAMA-first,
- ENHSP – Excesive Numeric Heuristic Search Planner.

Wszystkie 3 planery wygenerowały taki sam plan.

4.5. Wyniki eksperymentów

Planer	Liczba akcji
BFWS	5
LAMA	5
ENHSP	5

Tabela 1: Porównanie wyników działania planerów w zadaniu 2.

Wygenerowany plan prezentuje się następująco:

```
;;!domain: room-duster
;;!problem: clean-rooms

0.00000: (clean odkurzacz pokoj1)
0.00100: (move odkurzacz pokoj1 pokoj2)
0.00200: (clean odkurzacz pokoj2)
0.00300: (move odkurzacz pokoj2 pokoj3)
0.00400: (clean odkurzacz pokoj3)

; Makespan: 0.004
; Metric: 0.004
```

Wygenerowany plan składa się z sekwencji ruchów robota pomiędzy pokojami oraz akcji sprzątania pokoju, w którym aktualnie znajduje się robot. Planer dąży do osiągnięcia celu poprzez odwiedzenie wszystkich pomieszczeń i zmianę ich stanu z `dirty` na `clean`.

W tym prostym modelu każdy pokój jest połączony z każdym, więc plan w każdym przypadku zawiera minimalną liczbę ruchów. Gdyby urozmaicić domenę i dodać predykat `(path ?r1 - room ?r2 room)` definiujący istniejące przejście pomiędzy poszczególnymi pokojami, sytuacja mogłaby wyglądać inaczej – różne planery mogłyby zwracać różne plany (jak w zadaniu 3). Jednak treść zadania jasno narzucała predykaty modelu, więc zaimplementowałam wersję prostą.

5. Zadanie 3

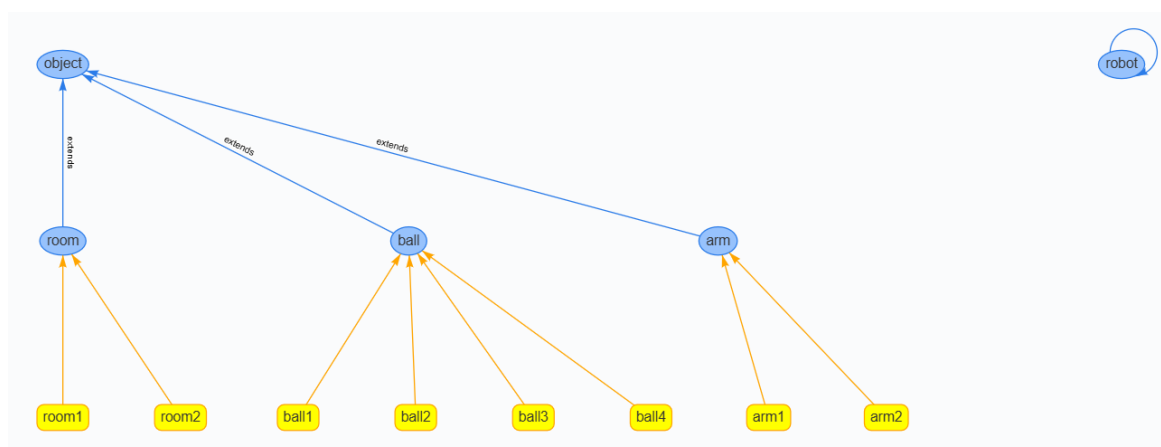
5.1. Opis problemu

Celem zadania było zamodelowanie problemu planowania, w którym robot wyposażony w dwa ramiona ma przenieść wszystkie piłki z pokoju **room1** do pokoju **room2**. Robot może poruszać się pomiędzy pokojami, podnosić piłki przy użyciu wolnych ramion oraz odkładać je w wybranym pomieszczeniu.

Aby wykonać zadanie, przeniosłam podany kod do odpowiednich plików i uruchomiłam planer.

5.2. Hierarchia obiektów

Poniżej znajduje się diagram hierarchii obiektów wykorzystanych w modelu.



Rysunek 4: Hierarchia obiektów w zadaniu 3.

5.3. Uruchomione planery

Uruchomiłam model z wykorzystaniem trzech różnych planerów:

- BFWS – FF-parser version,
- LAMA-first,
- ENHSP – Excessive Numeric Heuristic Search Planner.

Najlepszy wynik uzyskałam przy użyciu planera **LAMA-first** – wygenerowane rozwiązanie jest najbardziej optymalne z wszystkich możliwych rozwiązań. Pozostałe planery również znalazły poprawne rozwiązania (identyczne), jednak wygenerowane plany były nieoptymalne.

5.4. Wyniki eksperymentów

Planer	Liczba akcji
BFWS	15
LAMA	11
ENHSP	15

Tabela 2: Porównanie wyników działania planerów w zadaniu 3.

Plan wygenerowany przez planer LAMA prezentuje się następująco:

```
;;!domain: ball-moving-robot
;;!problem: move-balls

0.00000: (pick-up robot arm1 ball1 room1)
0.00100: (pick-up robot arm2 ball2 room1)
0.00200: (move robot room1 room2)
0.00300: (put-down robot arm1 ball1 room2)
0.00400: (put-down robot arm2 ball2 room2)
0.00500: (move robot room2 room1)
0.00600: (pick-up robot arm1 ball3 room1)
0.00700: (pick-up robot arm2 ball4 room1)
0.00800: (move robot room1 room2)
0.00900: (put-down robot arm1 ball3 room2)
0.01000: (put-down robot arm2 ball4 room2)

; Makespan: 0.010000000000000002
; Metric: 0.010000000000000002
```

Wygenerowany plan wykorzystuje oba ramiona robota jednocześnie, dzięki czemu możliwe było przetransportowanie dwóch piłek podczas jednego przejazdu między pokojami. Pozwoliło to ograniczyć liczbę ruchów robota oraz skrócić całkowity czas wykonania planu.

Z kolei planery BFWS oraz ENHSP wygenerowały następujący plan:

```
;;!domain: ball-moving-robot
;;!problem: move-balls

0.00000: (PICK-UP ROBOT ARM2 BALL1 ROOM1)
0.00100: (MOVE ROBOT ROOM1 ROOM2)
0.00200: (PUT-DOWN ROBOT ARM2 BALL1 ROOM2)
0.00300: (MOVE ROBOT ROOM2 ROOM1)
0.00400: (PICK-UP ROBOT ARM2 BALL2 ROOM1)
0.00500: (MOVE ROBOT ROOM1 ROOM2)
0.00600: (PUT-DOWN ROBOT ARM2 BALL2 ROOM2)
0.00700: (MOVE ROBOT ROOM2 ROOM1)
```

```
0.00800: (PICK-UP ROBOT ARM1 BALL4 ROOM1)
0.00900: (MOVE ROBOT ROOM1 ROOM2)
0.01000: (PUT-DOWN ROBOT ARM1 BALL4 ROOM2)
0.01100: (MOVE ROBOT ROOM2 ROOM1)
0.01200: (PICK-UP ROBOT ARM2 BALL3 ROOM1)
0.01300: (MOVE ROBOT ROOM1 ROOM2)
0.01400: (PUT-DOWN ROBOT ARM2 BALL3 ROOM2)

; Makespan: 0.014000000000000005
; Metric: 0.014000000000000005
```

W tym przypadku wygenerowany plan jest wyraźnie mniej efektywny. Plan nie wykorzystuje w pełni możliwości dwóch ramion robota. Przenoszone są pojedyncze piłki, co skutkuje większą liczbą przejazdów pomiędzy pokojami. Prowadzi to do wydłużenia całego czasu wykonania planu i zwiększenia ilości wykorzystanych zasobów. Choć rozwiązanie to jest teoretycznie poprawne, jest zupełnie nieoptymalne.

6. Podsumowanie

W ramach tej listy nauczyłam się modelowania problemów planowania w języku PDDL oraz korzystania z różnych planerów do generowania i analizy planów. Przygotowałam modele dla trzech różnych problemów, zaimplementowałam je w postaci plików `domain.pddl` i `problem.pddl`, a następnie uruchomiłam je w kilku planerach i porównałam wyniki. Dzięki temu zobaczyłam, jak różne podejścia wpływają na jakość i optymalność wygenerowanych planów.

7. Materiały źródłowe

1. Materiały do kursu
2. <https://fareskalaboud.github.io/LearnPDDL/>